Image Processing

Contents

Introduction.

Any image processing operation transforms the grey values of the pixels. However; image processing operations may be divided into in to three classes based on the information required to perform the transformation.

Introduction.

- From the most complex to the simplest; they are:
- Transforms. A transform represents the pixel values in some other, but equivalent form. Transforms allow for some very efficient and powerful algorithms.
- We may consider that in using a transform, the entire image is processed as a single large block.

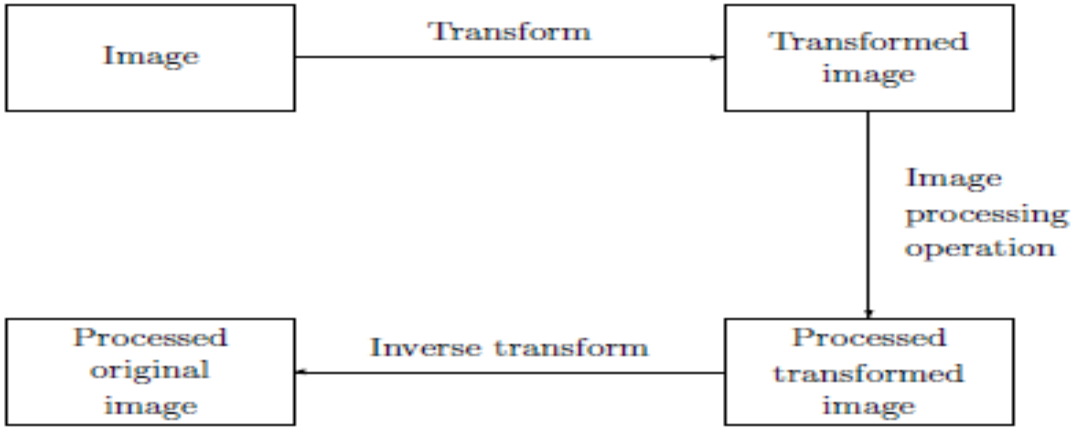


Figure 2.1: Schema for transform processing

Introduction.

- **Neighborhood processing: To change the grey level of a given pixel we need only know the value of the grey levels in a small neighborhood of pixels around the given pixel.**
- **Point operations: A pixel's grey value is changed without any knowledge of its surrounds.**

Arithmetic operations

- These operations act by applying simple function
- to each grey value in the image. thus f(x) is a function which maps the range 0-255 onto itself.

$$y = x \pm C$$

- Simple functions include adding or subtract a constant value to each pixel:

$$y = f(x)$$

- or multiplying each pixel by a constant:

$$y = Cx.$$

- we may have to fiddle the output slightly in order to ensure that the results are integers in the 0-255

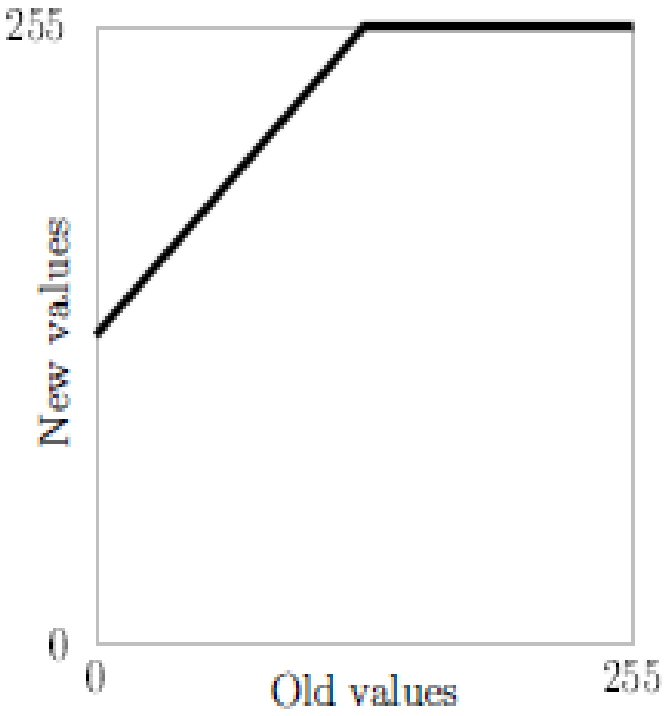
Arithmetic operations-Cont.

- We can do this by first rounding the result (if necessary) to obtain an integer; and then clipping the values by setting:

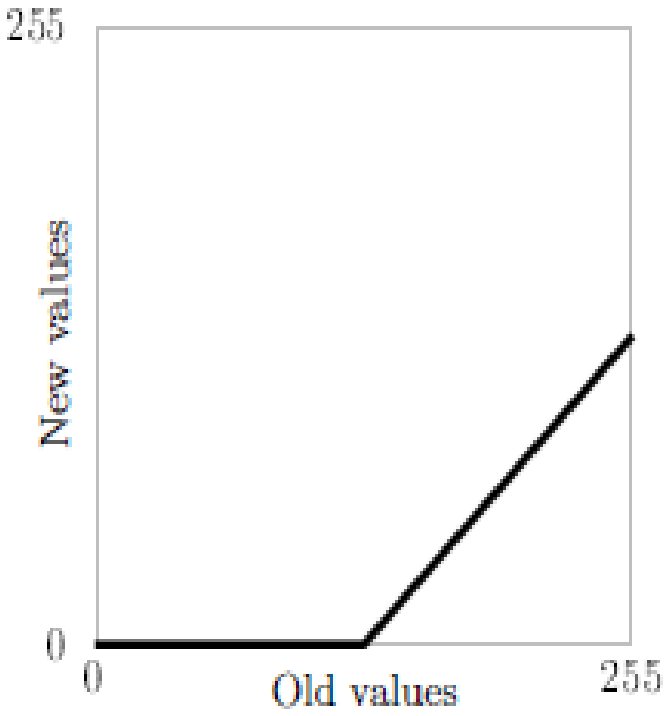
$$y \leftarrow \begin{cases} 255 & \text{if } y > 255, \\ 0 & \text{if } y < 0. \end{cases}$$

- We can obtain an understanding of how these operations affect an image by plotting y=f(x).
- The result of adding or subtracting 128 from each pixel in the image.

Arithmetic operations-Cont.



Adding 128 to each pixel



Subtracting 128 from each pixel

Arithmetic operations-Cont.

- when we add 128; all grey values of 127 or greater will be mapped to 255. And when we subtract 128; all grey values of 128 or less will be mapped to 0. By looking at these graphs; we observe that in general adding a constant will lighten an image; and subtracting a constant will darken it.
- >> b=imread('blocks.tif');
- >> whos b
- Name Size Bytes Class
- b 256x25 65536 uint8 array
- we can't perform arithmetic operations. If we try; we just get an error message:
- >> b1=b+128
- ??? Error using ==> +
- Function '+' not defined for variables of class 'uint8'.

Arithmetic operations-Cont.

- We can get round this in two ways. We can rst turn b into a matrix of type double; add the 128;
- and then turn back to uint8 for display:
- >> b1=uint8(double(b)+128);
- A second; and more elegant way; is to use the Matlab function imadd which is designed precisely
- to do this:
- >> b1=imadd(b;128);
- Subtraction is similar; we can transform out matrix in and out of double; or use the imsubtract
- function:
- >> b2=imsubtract(b;128);
- And now we can view them:
- >> imshow(b1);figure;imshow(b2)

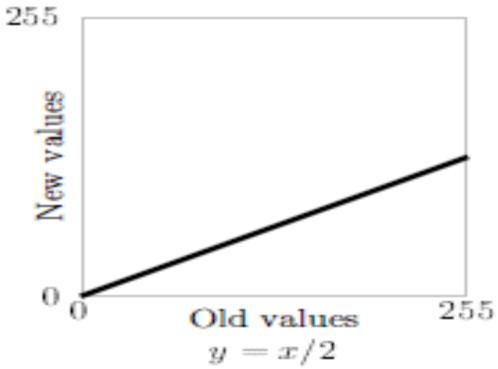
Arithmetic operations-Cont.

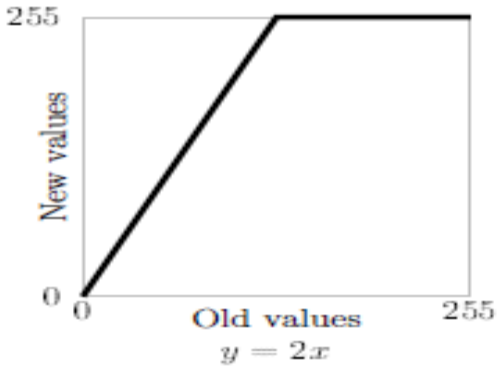
b1: Adding 128

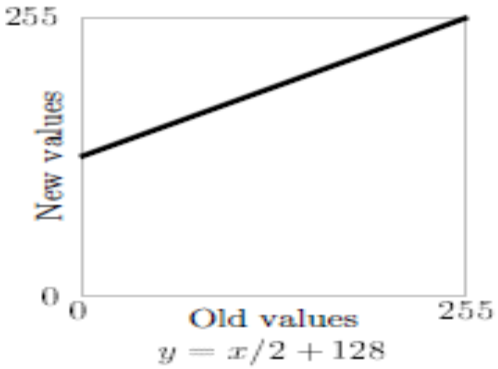
b2: Subtracting 128

Arithmetic operations-Cont.

- We can also perform lightening or darkening of an image by multiplication.







- To implement these functions; we use the immultiply function.

$y = x/2$	`b3=immultiply(b,0.5); or b3=imdivide(b,2)`
$y = 2x$	`b4=immultiply(b,2);`
$y = x/2 + 128$	`b5=imadd(immultiply(b,0.5),128); or b5=imadd(imdivide(b,2),128);`

the particular commands required to implement the functions

Arithmetic operations-Cont.

b3: $y = x/2$

b4: $y = 2x$

b5: $y = x/2 + 128$

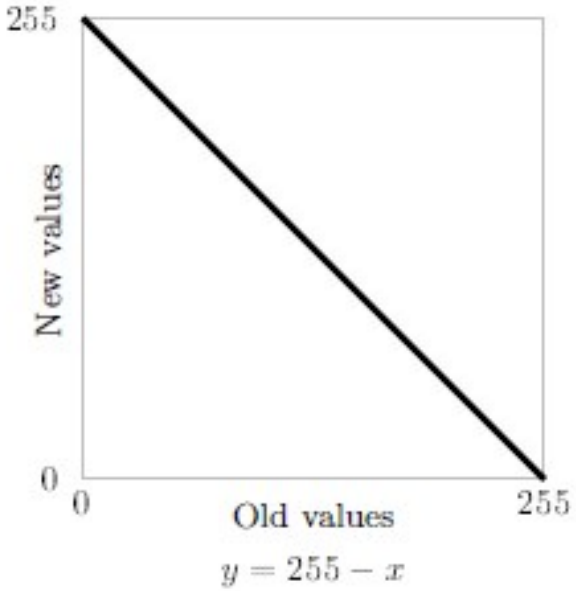
Arithmetic operations on an image: multiplication and division

Arithmetic operations-Cont.

Complements:

- The complement of a greyscale image is its photographic negative. If an image matrix m is of type double and so its grey values are in the range 0.0. to 1.0.
- we can obtain its negative with the command
- >> 1-m
- If the image is binary,we can use
- >> ~m
- If the image is of type uint8; the best approach is the imcomplement function.
- the complement function y-255-x and the result of the commands
- >> bc=imcomplement(b);
- >> imshow(bc)

Arithmetic operations-Cont.



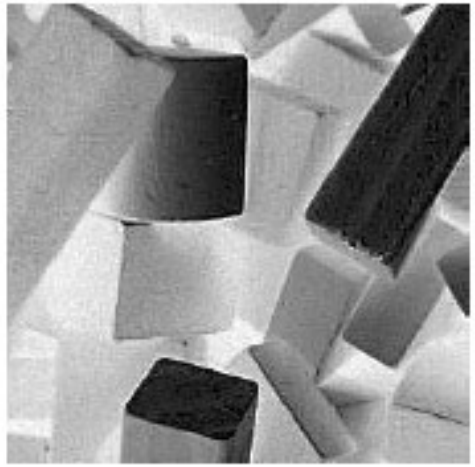
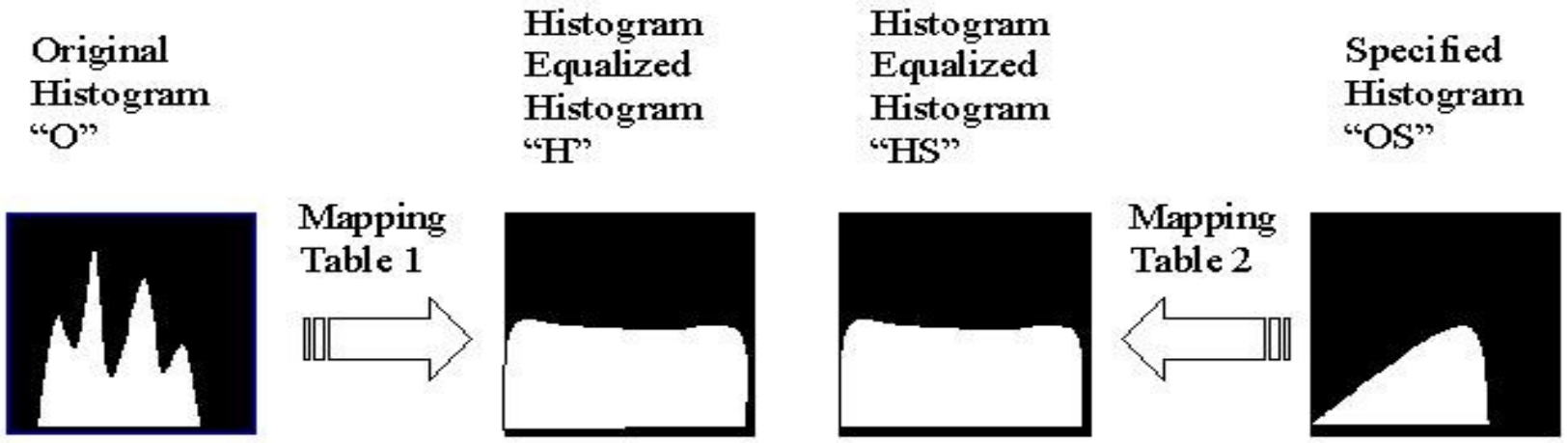


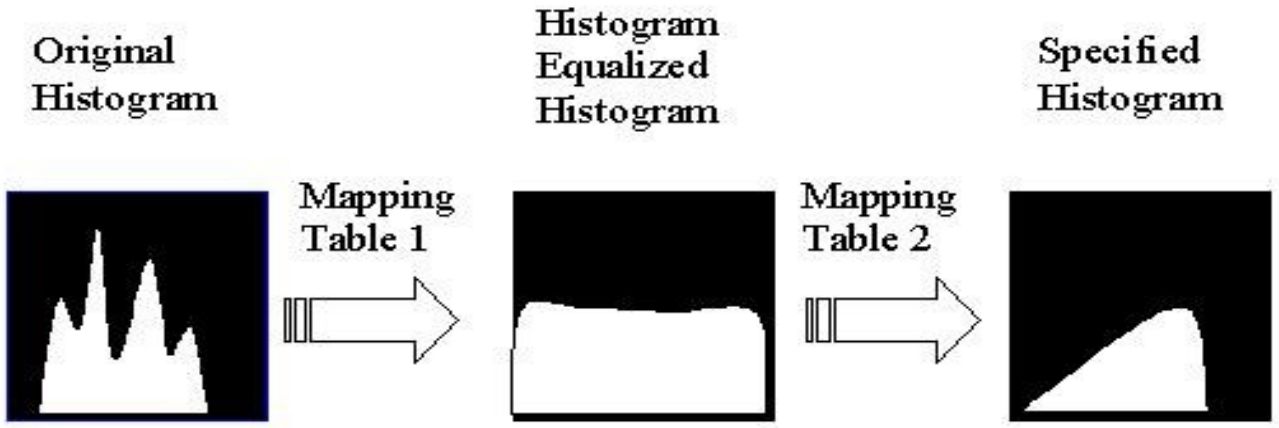
Image complementation

Histogram Specification. This figure is a conceptual look at histogram specification. a) Here we depict the histogram equalized versions of the original image histogram and the specified histogram. Now we have a common histogram for both, the histogram equalized version should both be approximately flat. b) Now we can use the histogram equalization mapping tables to get from the original histogram to the specified histogram

a



b



Histogram specification consists of following 5 steps:

1. Specify the desired histogram
2. Find the mapping table to histogram equalize the image, Mapping Table 1,
3. Find the mapping table to histogram equalize the values of the specified histogram, Mapping Table 2

Histogram specification steps (b continued)

4. Use mapping Tables 1 & 2 to find the mapping table to map the original values to the histogram equalized values and then to the specified histogram values
5. Use the table from step (4) to map the original values to the specified histogram values

EXAMPLE

1) Specify the desired histogram:

Gray Level Value	Number of pixels in desired histogram
0	1
1	5
2	10
3	15
4	20
5	0
6	0
7	0

2) For this we will use the image and mapping table from the previous example, where the histogram equalization mapping table (Mapping Table 1) is given by:

Original Gray Level Value level values-OS	Histogram Equalized equalized values-HS
0	1
1	2
2	4
3	4
4	6
5	6
6	7
7	7

3) Find the histogram equalization mapping table (Mapping Table 2) for the specified histogram:

Gray Level	Histogram Equalized
Value - OS	Values – HS .
0	round(1/51)*7 = 0
1	round(6/51)*7 = 1
2	round(16/51)*7 = 2
3	round(31/51)*7 = 4
4	round(51/51)*7 = 7
5	round(51/51)*7 = 7
6	round(51/51)*7 = 7
7	round(51/51)*7 = 7

4) Use Mapping Tables 1 and 2 to find the final mapping table by mapping the values first to the histogram equalized values and then to the specified histogram values.

Mapping Table 1

O	H
0	1
1	2
2	4
3	4
4	6
5	6
6	7
7	7

Mapping Table 2

HS	M
0	1
1	2
2	3
4	3
7	4
7	4
7	4
7	4

5) Use the table from STEP 4 to perform the histogram specification mapping. For this all we need are columns O (or OS) and M:

O	M
0	1
1	2
2	3
3	3
4	4
5	4
6	4
7	4

Now, all the 0′s get mapped to 1′s, the 1′s to 2′s, the 3′s to 3′s and so on

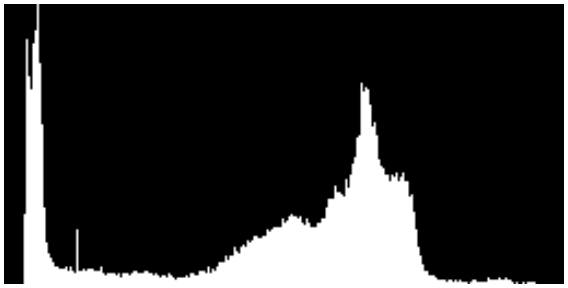
- In practice, the desired histogram is often specified by a continuous (possibly non-linear) function, for example a sine or a log function.

- To obtain the numbers for the specified histogram the function is sampled, the values are normalized to 1, and then multiplied by the total number of pixels in the image

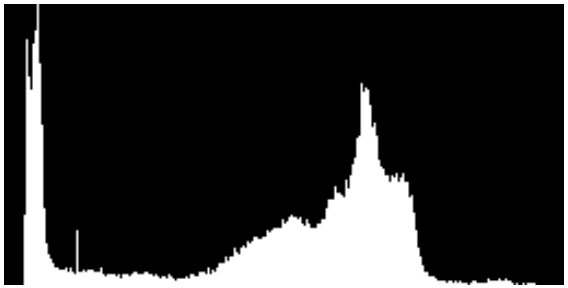
Histogram Specification Examples

Original image

Histogram of original image

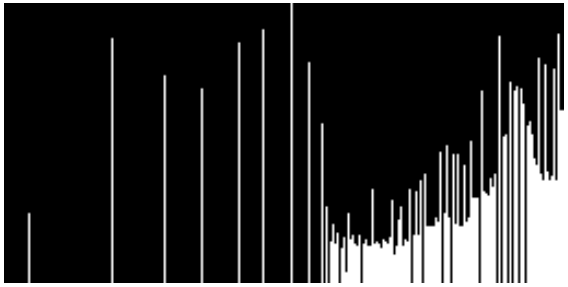


Histogram Specification Examples cont'd



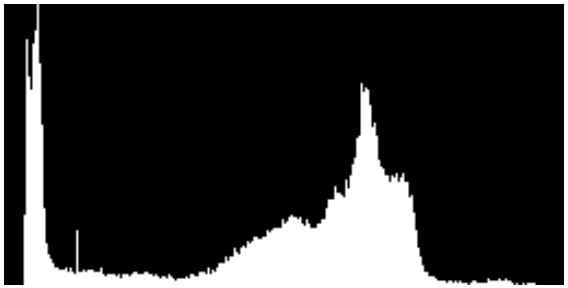
Original histogram

Specified histogram, *exp(0.015*x)*



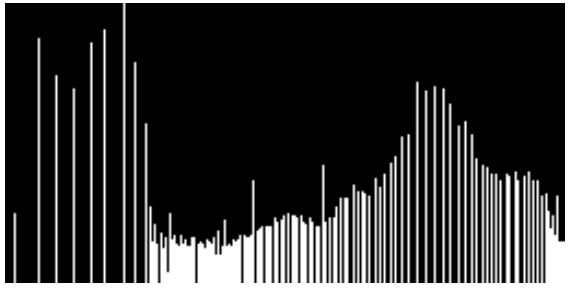
Output image and its histogram

Histogram Specification Examples cont'd



Original histogram

Specified histogram, *log(0.5*x+2)*



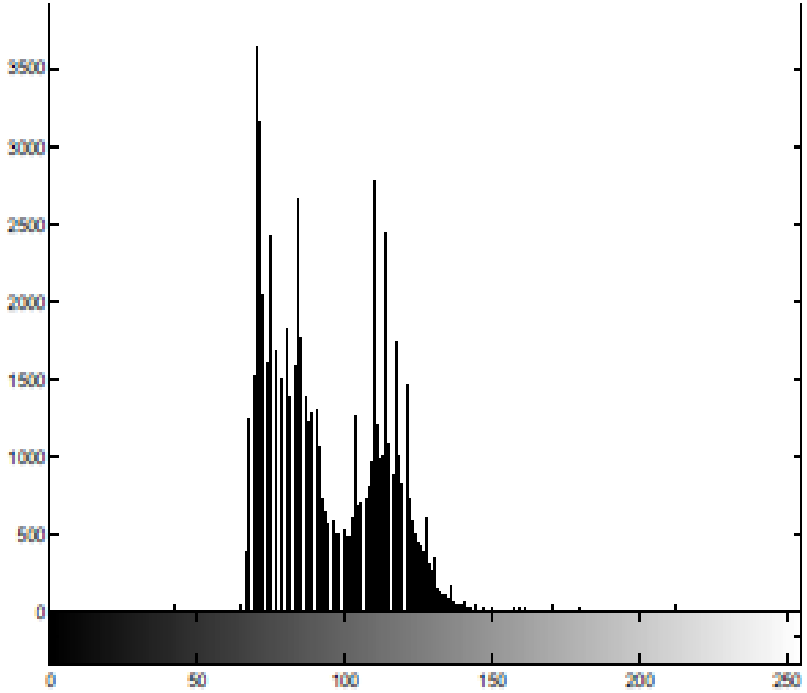
Output image and its histogram

Histograms

- Given a greyscale image, its histogram consists of the histogram of its grey levels, that is, a graph indicating the number of times each grey level occurs in the image. We can infer a great deal about the appearance of an image from its histogram, as the following examples indicate:
  - In a dark image, the grey levels (and hence the histogram) would be clustered at the lower end.
  - In a uniformly bright image,the grey levels would be clustered at the upper end:
  - In a well contrasted image,the grey levels would be well spread out over much of the range.

Histograms-cont.

- (the axis tight command ensures the axes of the histogram are automatically scaled to t all the values in).



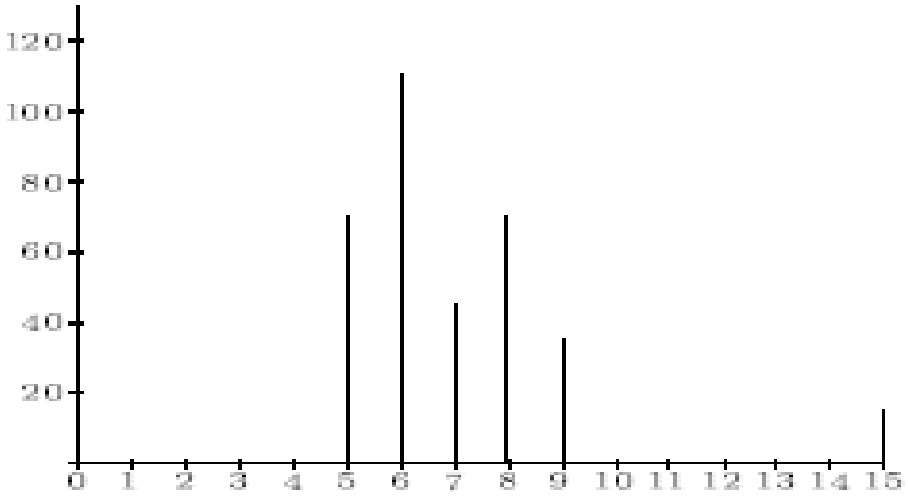
Histograms-cont.

- We can view the histogram of an image in Matlab by using the imhist function:

```
>> p=imread('pout.tif');
>> imshow(p);figure;imhist(p);axis tight
```

Histogram stretching (Contrast stretching)

Suppose we have an image with the histogram

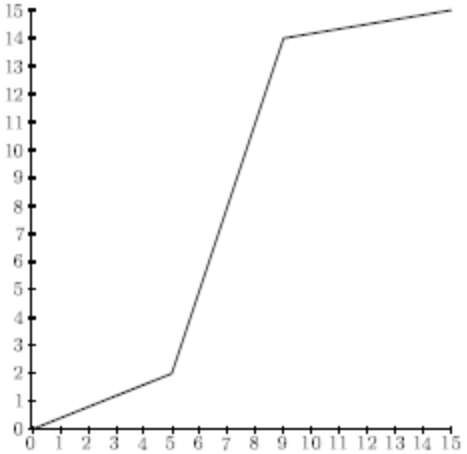


Histograms-cont.

- associated with a table of the Numbers ni of grey values:

Grey level i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
n_i	15	0	0	0	0	70	110	45	70	35	0	0	0	0	0	15

- (with N=360; as before.) We can stretch the grey levels in the centre of the range out by applying the piecewise linear function.



Histograms-cont.

- This function has the effect of stretching
- the grey levels 5-9 to grey levels 2-14 according to the equation:

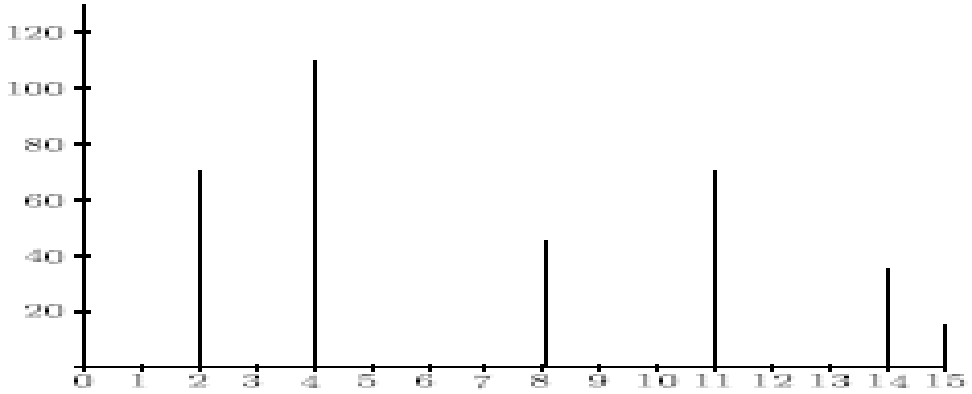
$$j = \frac{14-2}{9-5}(i-5)+2$$

- where i is the original grey level and j its result after the transformation. Grey levels outside this range are either left alone (as in this case) or transformed according to the linear functions at the ends of the graph above. This yields:

i	5	6	7	8	9
j	2	5	8	11	14

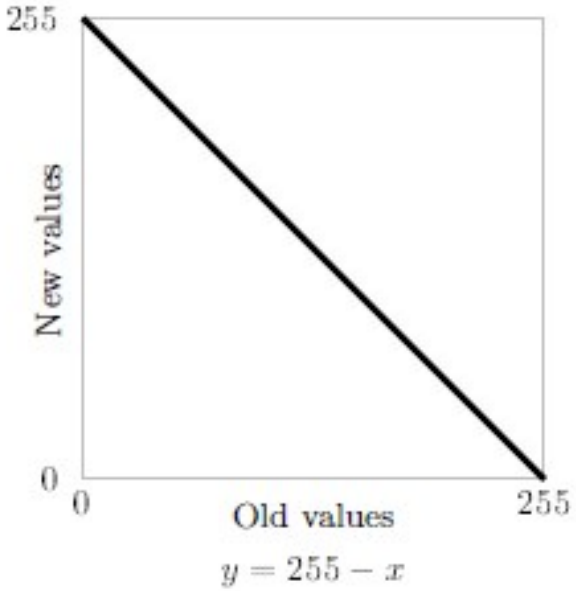
Histograms-cont.

- and the corresponding histogram:



- which indicates an image with greater contrast than the original.
- Use of imadjust To perform histogram stretching in Matlab the imadjust function may be used. In its simplest incarnation, the command imadjust(im;[a;b];[c;d]) stretches the image according to the function

Arithmetic operations-Cont.



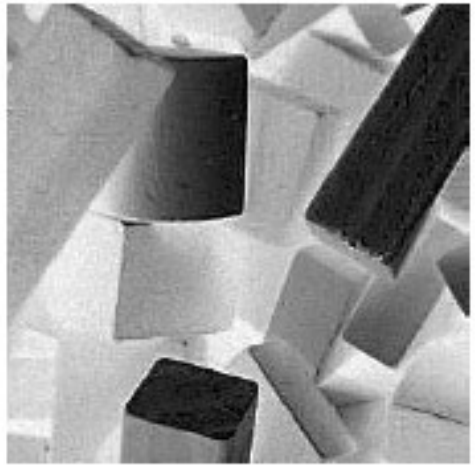
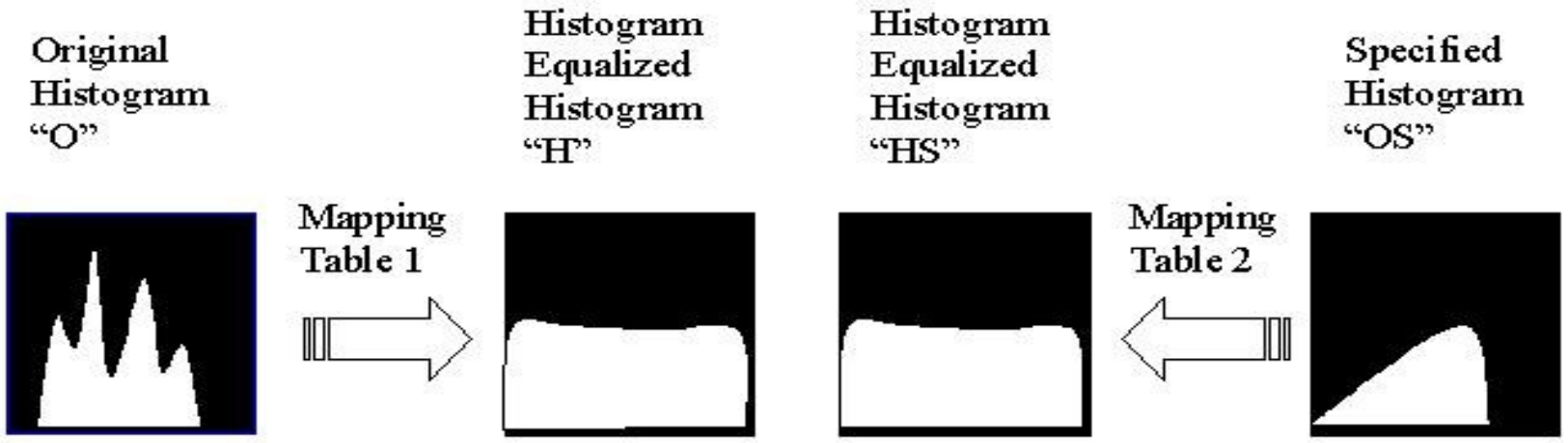


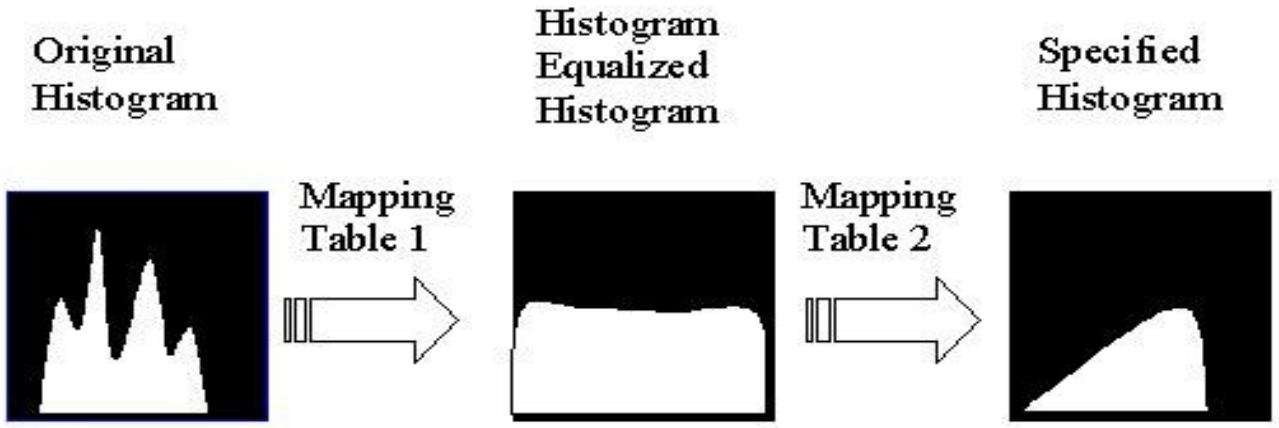
Image complementation

Histogram Specification. This figure is a conceptual look at histogram specification. a) Here we depict the histogram equalized versions of the original image histogram and the specified histogram. Now we have a common histogram for both, the histogram equalized version should both be approximately flat. b) Now we can use the histogram equalization mapping tables to get from the original histogram to the specified histogram

a



b



Histogram specification consists of following 5 steps:

1. Specify the desired histogram
2. Find the mapping table to histogram equalize the image, Mapping Table 1,
3. Find the mapping table to histogram equalize the values of the specified histogram, Mapping Table 2

Histogram specification steps (b continued)

4. Use mapping Tables 1 & 2 to find the mapping table to map the original values to the histogram equalized values and then to the specified histogram values
5. Use the table from step (4) to map the original values to the specified histogram values

EXAMPLE

1) Specify the desired histogram:

Gray Level Value	Number of pixels in desired histogram
0	1
1	5
2	10
3	15
4	20
5	0
6	0
7	0

2) For this we will use the image and mapping table from the previous example, where the histogram equalization mapping table (Mapping Table 1) is given by:

Original Gray Level Value level values-OS	Histogram Equalized equalized values-HS
0	1
1	2
2	4
3	4
4	6
5	6
6	7
7	7

3) Find the histogram equalization mapping table (Mapping Table 2) for the specified histogram:

Gray Level	Histogram Equalized
Value - OS	Values – HS .
0	round(1/51)*7 = 0
1	round(6/51)*7 = 1
2	round(16/51)*7 = 2
3	round(31/51)*7 = 4
4	round(51/51)*7 = 7
5	round(51/51)*7 = 7
6	round(51/51)*7 = 7
7	round(51/51)*7 = 7

4) Use Mapping Tables 1 and 2 to find the final mapping table by mapping the values first to the histogram equalized values and then to the specified histogram values.

Mapping Table 1

O	H
0	1
1	2
2	4
3	4
4	6
5	6
6	7
7	7

Mapping Table 2

HS	M
0	1
1	2
2	3
4	3
7	4
7	4
7	4
7	4

5) Use the table from STEP 4 to perform the histogram specification mapping. For this all we need are columns O (or OS) and M:

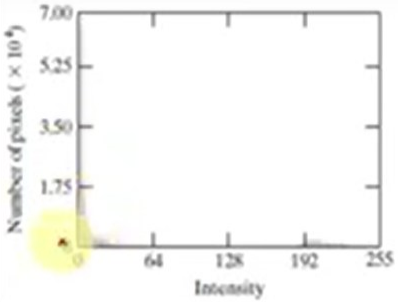
O	M
0	1
1	2
2	3
3	3
4	4
5	4
6	4
7	4

Now, all the 0′s get mapped to 1′s, the 1′s to 2′s, the 3′s to 3′s and so on

❖ Histogram Matching/Specification:

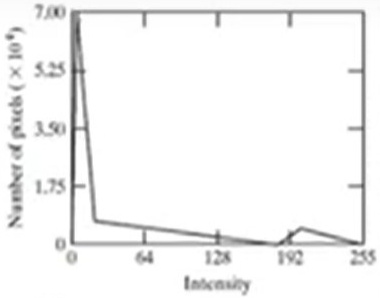
- Histogram equalization is automatic and is not always good.
- At many times it is desirable to have an interactive method in which certain grey levels are highlighted.
- If we modify the grey level of an image that has a uniform PDF, using the inverse transformation we can get back the original histogram. Using this knowledge, we can obtain any shape of the grey level histogram by processing the given image.
- The method used to generate a processed image that has a specified histogram is called histogram matching or specification.

FIGURE 3.23
(a) Image of the Mars moon Phobos taken by NASA's *Mars Global Surveyor*. (b) Histogram. (Original image courtesy of NASA.)

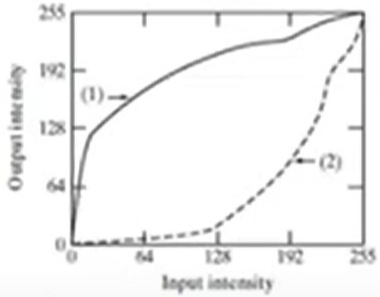


a c
b
d

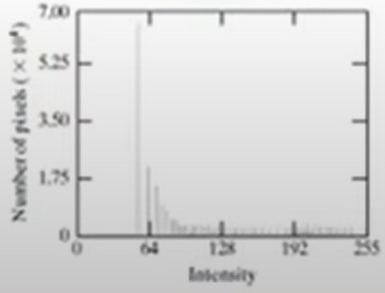
FIGURE 3.25
(a) Specified histogram. (b) Transformations. (c) Enhanced image using mappings from curve (2). (d) Histogram of (c).

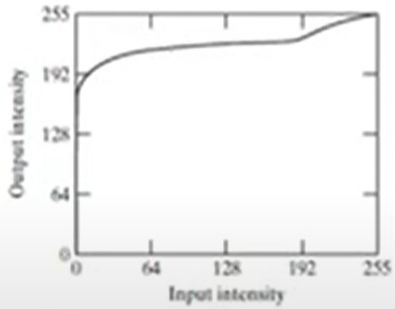


$G(z_q) = s_k$



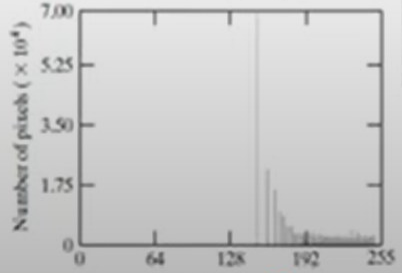
$z_q = G^{-1}(s_k)$





a b
c

FIGURE 3.24
(a) Transformation function for histogram equalization. (b) Histogram-equalized image (note the washed-out appearance). (c) Histogram of (b).



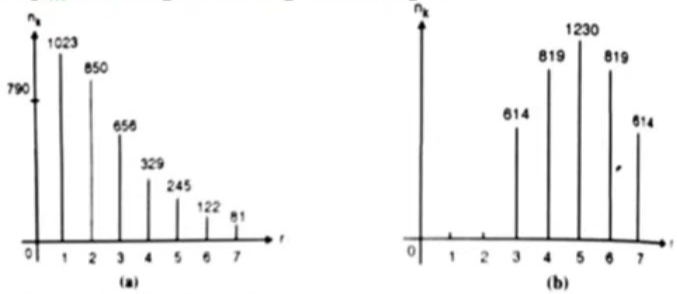
Grey Level	0	1	2	3	4	5	6	7
No. of pixels	790	1023	850	656	329	245	122	81

Image A

Grey Level	0	1	2	3	4	5	6	7
No. of pixels	0	0	0	614	819	1230	819	614

Image B

Step 1: Plot histogram for image A and Image B



Step 2: Equalize image A

Grey Level	n_k	$P_r(r_k) = n_k/n$	CDF	CDF x (L-1) L=8	New Grey Level	New Pixel Value
0	790	0.19	0.19	1.33	1	790
1	1023	0.25	0.44	3.08	3	1023
2	850	0.21	0.65	4.55	5	850
3	656	0.16	0.81	5.67	6	656 +
4	329	0.08	0.89	6.23	6	329 =985
5	245	0.06	0.95	6.25	7	245 +
6	122	0.03	0.98	6.86	7	122 + 81
7	81	0.02	1	7	7	= 448
Total (n)	4096					

Step 3: Equalize image B

Before equalizing image B, note the values of the distinct histogram level... histogram equalization of image A.

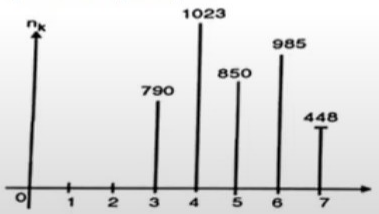
Grey Level	0	1	2	3	4	5	6	7
No. of pixels	0	790	0	1023	0	850	985	448

Now equalize image B

Grey Level	n_k	$P_r(r_k) = n_k/n$	CDF	CDF x (L-1) L=8	Rounding off	New Grey Level
0	0	0	0	0	0	0
1	0	0	0	0	0	0
2	0	0	0	0	0	0
3	614	0.149	0.149	1.05	1	1
4	819	0.20	0.35	2.5	3	3
5	1230	0.30	0.65	4.55	5	5
6	819	0.20	0.85	5.97	6	6
7	614	0.15	1	7	7	7
Total(n)	4096					

Grey level	Equalization values of Problem Image	Equalization values of Reference Image	Mapping For Histogram Specification
0	1	0	3
1	3	0	4
2	5	0	5
3	6	1	6
4	6	3	6
5	7	5	7
6	7	6	7
7	7	7	7

Step 4: Plotting the resulting histogram



Resultant histogram

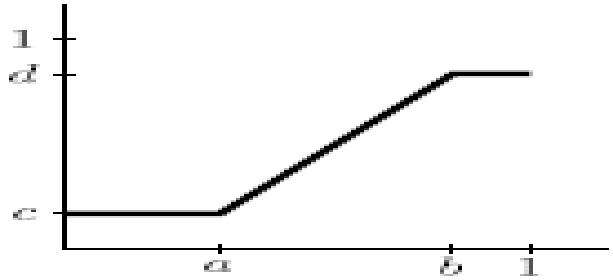
Grey Level	0	1	2	3	4	5	6	7
No. of pixels	0	0	0	790	1023	850	985	448

- In practice, the desired histogram is often specified by a continuous (possibly non-linear) function, for example a sine or a log function.

- To obtain the numbers for the specified histogram the function is sampled, the values are normalized to 1, and then multiplied by the total number of pixels in the image

Histograms-cont.

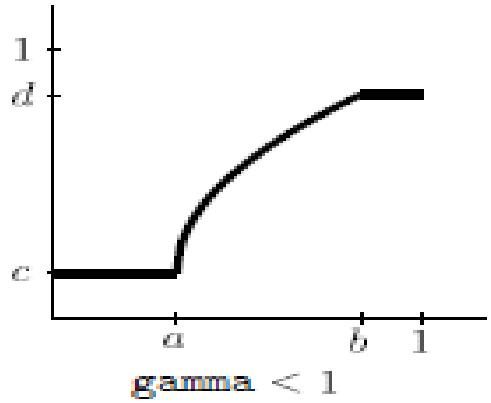
- The stretching function given by imadjust.



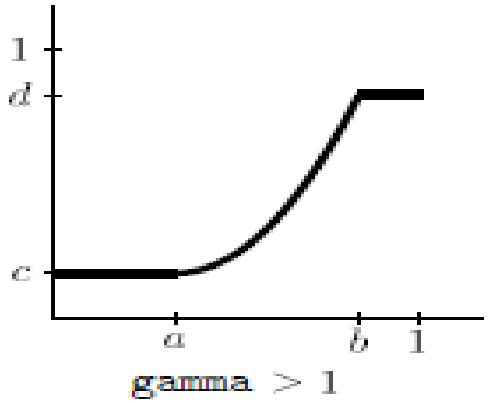
- >> imadjust(im;[];[])
- does nothing; and the command
- >> imadjust(im;[];[1;0])
- inverts the grey values of the image; to produce a result similar to a photographic negative.
- The imadjust function has one other optional parameter: the gamma value; which describes

Histograms-cont.

- the shape of the function between the coordinates (a;c) and (b;d).
  - If gamma is equal to 1; which is the default; then a linear mapping is used; as shown above in gure 2.10. However; values less than one produce a function which is concave downward



gamma < 1



gamma > 1

Histograms-cont.

- The function used is a slight variation on the standard line between two points: $y = \left(\frac{x-a}{b-a}\right)^{\gamma}(d-c)+c.$
- Use of the gamma value alone can be enough to substantially change the appearance of the image.
- For example:
- >> t=imread('tire.tif');
- >> th=imadjust(t;[];[];0.5);
- >> imshow(t);figure;imshow(th)

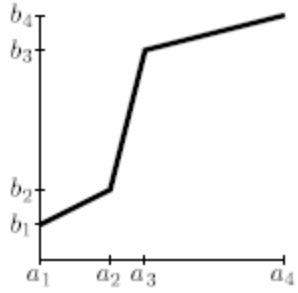
Histograms-cont.

- A piecewise linear stretching functionWe can easily write our own function to perform piecewise linear stretching.
- To do this; we will make use of the

find function; to find the pixel values in t between i and ai+1.

- Since the line between the

Coordinates (ai;bi) and (ai+1;bi+1) .



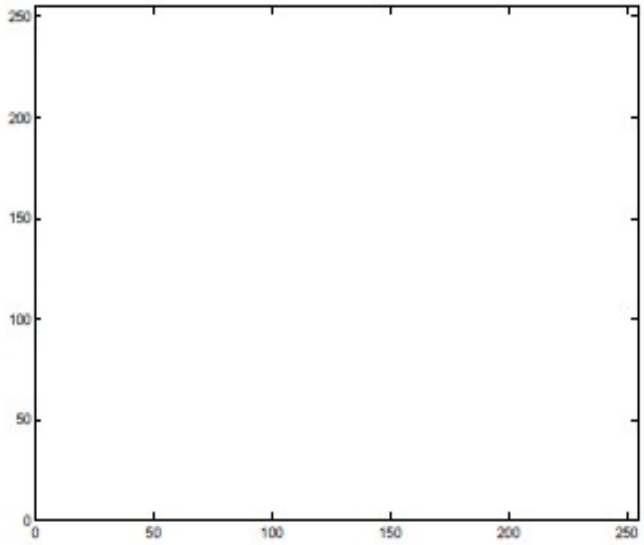
$$y = \frac{b_{i+1} - b_i}{a_{i+1} - a_i}(x - a_i) + b_i$$

Histograms-cont.

- the heart of our function will be the lines
- pix=find(im >= a(i) & im < a(i+1));
- out(pix)=(im(pix)-a(i))*(b(i+1)-b(i))/(a(i+1)-a(i))+b(i);
- where im is the input image and out is the output image. A simple procedure which takes as inputs
- images of type uint8 or double is shown in figure 2.15. As an example of the use of this function:

Histograms-cont.

- >> th=histpwl(t;[0 .25 .5 .75 1];[0 .75 .25 .5 1]);
- >> imshow(th)
- >> figure;plot(t;th;'.');axis tight



Histograms-cont.

```
function out = histpwl(im,a,b)
%
% HISTPWL(IM,A,B) applies a piecewise linear transformation to the pixel values
% of image IM, where A and B are vectors containing the x and y coordinates
% of the ends of the line segments.  IM can be of type UINT8 or DOUBLE,
% and the values in A and B must be between 0 and 1.
%
% For example:
%
%   histpwl(x,[0,1],[1,0])
%
% simply inverts the pixel values.
%
classChanged = 0;
if ~isa(im, 'double'),
     classChanged = 1;
     im = im2double(im);
end

if length(a) ~= length (b)
  error('Vectors A and B must be of equal size');
end

N=length(a);
out=zeros(size(im));

for i=1:N-1
  pix=find(im>=a(i) & im<a(i+1));
  out(pix)=(im(pix)-a(i))*(b(i+1)-b(i))/(a(i+1)-a(i))+b(i);
end

pix=find(im==a(N));
out(pix)=b(N);

if classChanged==1
  out = uint8(255*out);
end
```

- **A Matlab function for applying a piecewise linear stretching function**

Histograms-cont.

- **Histogram equalization:**
- **The trouble with any of the above methods of histogram stretching is that they require user input.**

  Sometimes a better approach is provided by histogram equalization, which is an entirely automatic procedure.
- **The idea is to change the histogram to one which is uniform; that is that every bar on the histogram is of the same height, or in other words that each grey level in the image occurs with**

  the same frequency.
- **Suppose our image has L deferent grey levels 0,1,2,..,L-1 and that grey level i occurs ni times in the image.**

Histograms-cont.

- Suppose also that the total number of pixels in the image is n so that .

$$n_0 + n_1 + n_2 + \cdots + n_{L-1} = n.$$

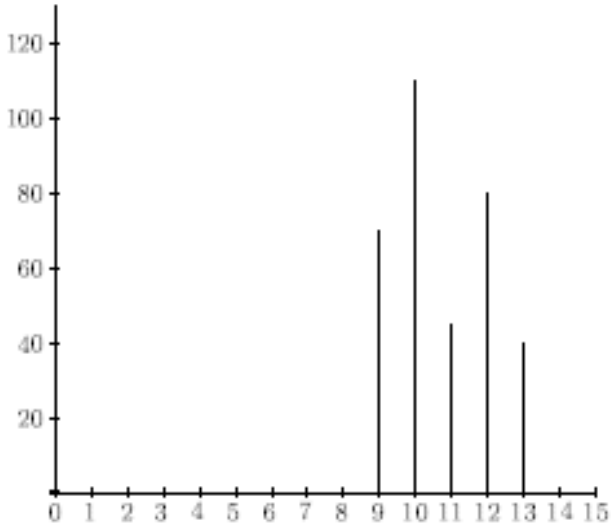
- To transform the grey levels to obtain a better contrasted image, we change grey level i

$$\left(\frac{n_0 + n_1 + \cdots + n_i}{n}\right)(L-1).$$

- and this number is rounded to the nearest integer.

Histograms-cont.

- Suppose a 4-bit grayscale image has the histogram.



- associated with a table of the number ni of grey values:

Grey level i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
n_i	15	0	0	0	0	0	0	0	0	70	110	45	80	40	0	0

Histograms-cont.

- We would expect this image to be uniformly bright, with a few dark dots on it. To equalize this histogram, we form running totals of the ni each by 15/360

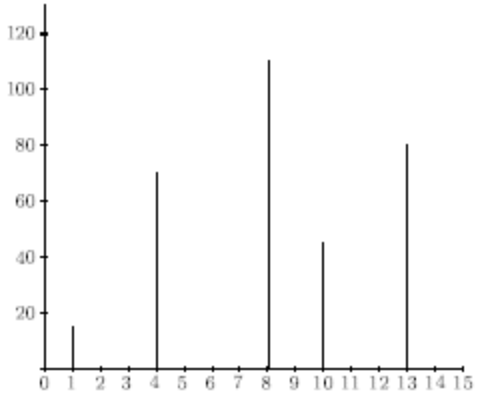
Grey level i	n_i	Σn_i	$(1/24)\Sigma n_i$	Rounded value
0	15	15	0.63	1
1	0	15	0.63	1
2	0	15	0.63	1
3	0	15	0.63	1
4	0	15	0.63	1
5	0	15	0.63	1
6	0	15	0.63	1
7	0	15	0.63	1
8	0	15	0.63	1
9	70	85	3.65	4
10	110	195	8.13	8
11	45	240	10	10
12	80	320	13.33	13
13	40	360	15	15
14	0	360	15	15
15	0	360	15	15

Histograms-cont.

- We now have the following transformation of grey values, obtained by reading o the first and last columns in the above table:

Original grey level i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Final grey level j	1	1	1	1	1	1	1	1	1	4	8	10	13	15	15	15

- and the histogram of the j values This is far more spread out than the original histogram, and so the resulting image should exhibit greater contrast.

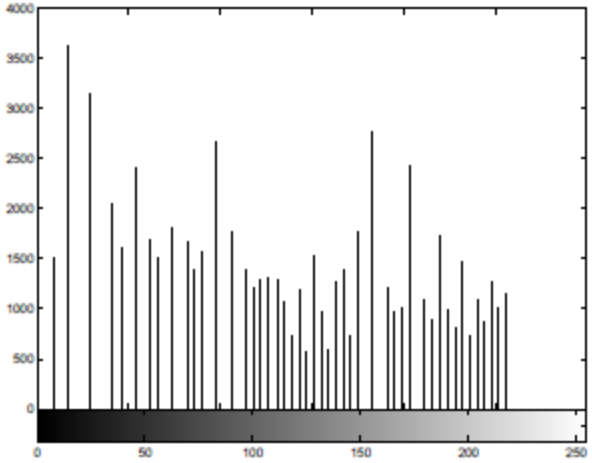


Histograms-cont.

- To apply histogram equalization in Matlab, use the histeq function; for example:

```
>> p=imread('pout.tif');
>> ph=histeq(p);
>> imshow(ph),figure,imhist(ph),axis tight
```

- applies histogram equalization to the pout image, and produces the resulting histogram.



Histograms-cont.

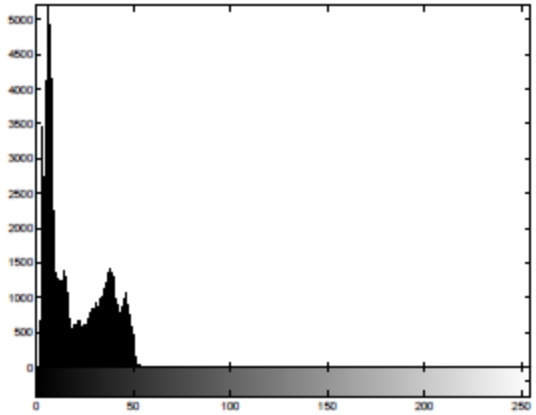
- Notice the far greater spread of the histogram. This corresponds to the greater increase of contrast in the image.
- We give one more example, that of a very dark image. We can obtain a dark image by taking an image and using imdivide.

```
>> en=imread('engineer.tif');
>> e=imdivide(en,4);
```

Histograms-cont.

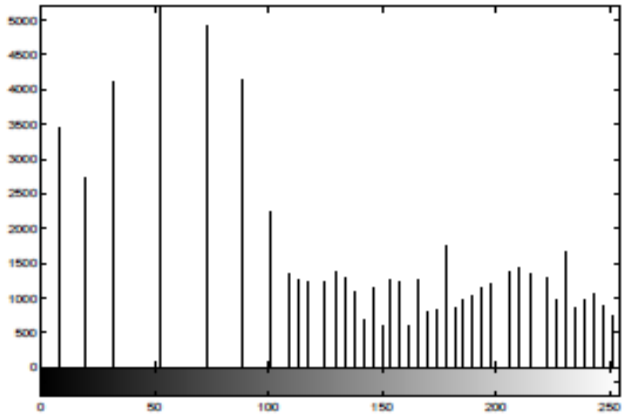
- Since the matrix e contains only low values it will appear very dark when displayed. We can display this matrix and its histogram with the usual commands:

>> imshow(e),figure,imhist(e),axis tight.



Histograms-cont.

- the very dark image has a corresponding histogram heavily clustered at the lower end of the scale.
- But we can apply histogram equalization to this image, and display the results:
- >> eh=histeq(e);
- >> imshow(eh),figure,imhist(eh),axis tight



Histograms-cont.

- **Lookup tables:**
- **Point operations can be performed very effectively by the use of a lookup table, known more simply as an LUT.**
- **For operating on images of type uint8, such a table consists of a single array of 256 values, each value of which is an integer in the range 0 -255 .**
- **Then our operation can be implemented by replacing each pixel value p by the corresponding value tp in the table.**
- **For example, the LUT corresponding to division by 2 looks like:**

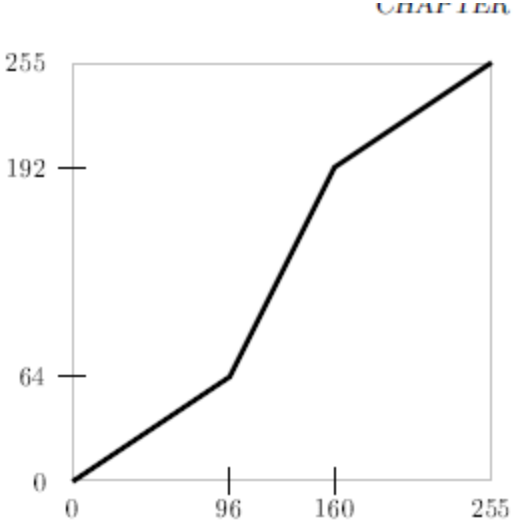
Index:	0	1	2	3	4	5	...	250	251	252	253	254	255
LUT:	0	0	1	1	2	2	...	125	125	126	126	127	127

Histograms-cont.

- This means, for example, that a pixel with value 4 will be replaced with 2; a pixel with value 253 will be replaced with value 126.
- If T is a lookup table in Matlab, and im is our image, the the lookup table can be applied by the simple command::
- T(im)
- For example, suppose we wish to apply the above lookup table to the blocks image. We can create
- the table with
- >> T=uint8(floor(0:255)/2);
- apply it to the blocks image b with
- >> b2=T(b);

Histograms-cont.

- The image b2 is of type uint8, and so can be viewed directly with imshow. As another example, suppose we wish to apply an LUT to implement the contrast stretching function



Histograms-cont.

- the equations of the three lines used are:

$$
\begin{aligned}
y &= \frac{64}{96}x, \\
y &= \frac{192-64}{160-96}(x-96)+64, \\
y &= \frac{255-192}{255-160}(x-160)+192,
\end{aligned}
$$

- and these equations can be written more simply as

$$
\begin{aligned}
y &= 0.6667x, \\
y &= 2x-128, \\
y &= 0.6632x+85.8947.
\end{aligned}
$$